
CYBERSECURITY ASSESSMENT REPORT

Web Application Security Testing – Week 1 Internship Task

Submitted By

Name: Shahbaz Iftikhar

Role: Cybersecurity Intern

Application Tested

OWASP Juice Shop

Testing Environment

- Kali Linux
- Node.js & npm
- OWASP Juice Shop
- OWASP ZAP
- Firefox Browser Developer Tool

Table of Content

1- Introduction.....	3
2- Application Setup.....	3
3- Tools Used.....	3
4- Testing Methodology.....	4
4.1 Automated Testing.....	4
5- Vulnerability Assessment Findings.....	4
5.1 Cross-Site Scripting (XSS).....	4
5.2 SQL Injection Testing.....	5
6- OWASP ZAP Scan Results.....	7
7- Recommendations.....	9
8. Conclusion.....	10

1- Introduction

This report documents the security assessment performed on the OWASP Juice Shop web application. The objective of the assessment was to identify common web application vulnerabilities such as Cross-Site Scripting (XSS), security misconfigurations, information disclosure issues, and weak security header implementations. The testing was conducted in a local environment using Kali Linux and OWASP ZAP.

2- Application Setup

The following steps were performed to set up the application:

1. Installed Node.js and npm on Kali Linux.
2. Cloned the OWASP Juice Shop repository from GitHub.
3. Installed dependencies using npm.
4. Started the application locally.
5. Accessed the application at:

<http://localhost:3000>

The application was successfully launched and tested in a controlled local environment.

3- Tools Used

Tool	Purpose
Kali Linux	Testing environment
OWASP Juice Shop	Vulnerable web application
OWASP ZAP	Automated vulnerability scanning
Browser Developer Tools	Manual testing and inspection
npm / Node.js	Application runtime and dependency management

4- Testing Methodology

4.1 Automated Testing

- Used OWASP ZAP Automatic Scan feature.
- Scanned the local application running on localhost:3000.
- Collected vulnerability alerts and security findings.

5- Vulnerability Assessment Findings

5.1 Cross-Site Scripting (XSS)

Description

Cross-Site Scripting (XSS) occurs when user input is not properly sanitized, allowing attackers to inject and execute malicious JavaScript code inside a victim's browser. I tried different scripts the

Payload Used

```
<script>alert('XSS');</script> this payload was not working  
so tried different payloads to check xss vulnerabilities  
I used <img src=x onerror= alert('xss')> and  
<iframe src="javascript:alert(1)">  
i tried these payloads and found xss vulnerabilities as  
shown in fig 1
```

Result

The payload executed successfully and generated a JavaScript alert box, indicating potential XSS behavior.

Risk

An attacker may execute malicious scripts, steal session information, redirect users, or manipulate webpage content.

Recommendation

- Validate and sanitize user input.

- Implement output encoding.
- Use Content Security Policy (CSP).

Screenshot Placeholder

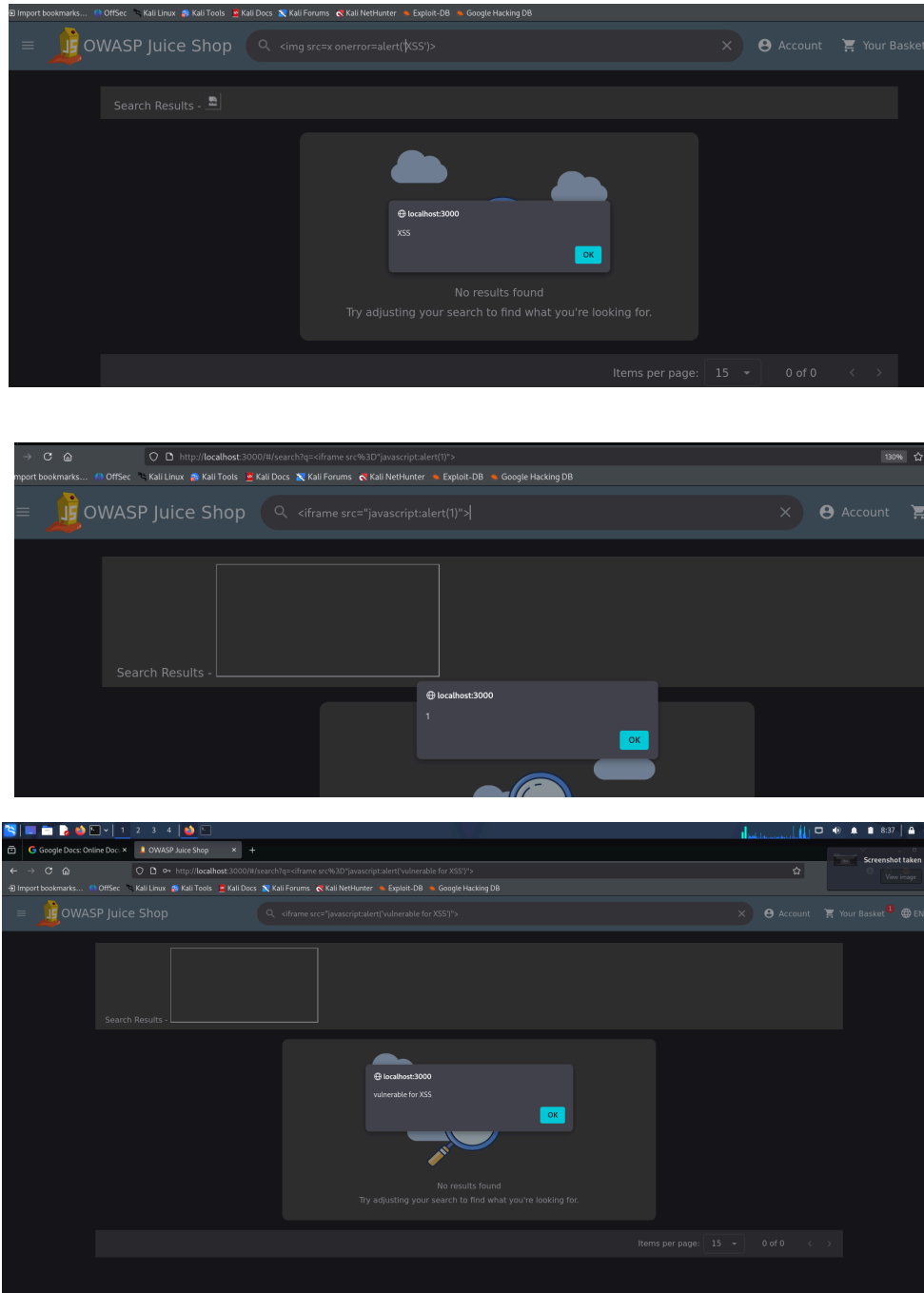


Fig 1.1 XSS vulnerabilities

5.2 SQL Injection Testing

Description

SQL Injection testing was performed on login and input fields using common SQL injection payloads.

Payload Example

admin' OR '1'='1 i tried it through different ways i used it as ' OR '1'='1 in password and username field then i successfully logged in i logged in as admin as shown in fig 2

Result

The application rejected the malicious input and displayed validation or error responses. Direct login bypass was not achieved.

Observation

Improper error handling was observed during testing, including responses such as:

[object Object]

This may indicate poor exception handling or information leakage.

Risk

Improper error handling can expose internal application behavior and assist attackers during reconnaissance.

Recommendation

- Use parameterized queries.
- Implement secure error handling.
- Avoid exposing raw objects or stack traces.

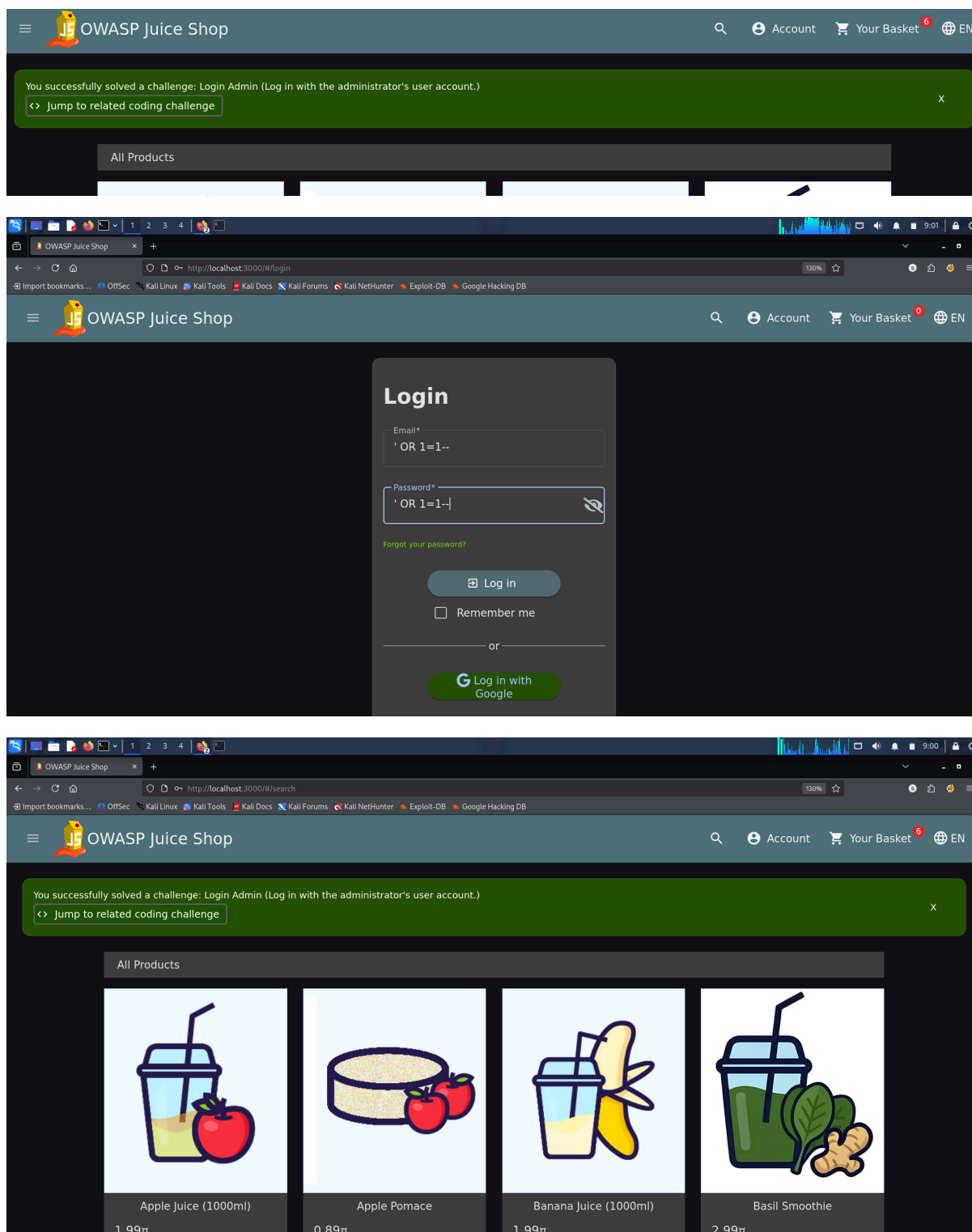


Fig 2 SQL injection

6- OWASP ZAP Scan Results

OWASP ZAP Automatic Scan was performed against:

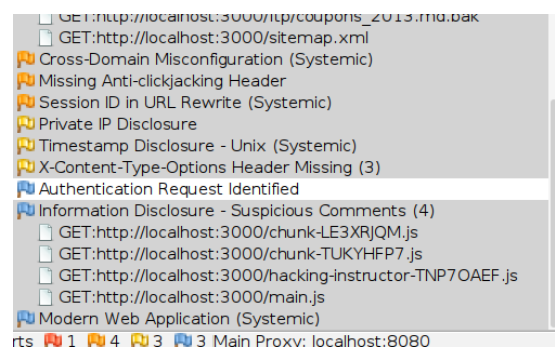
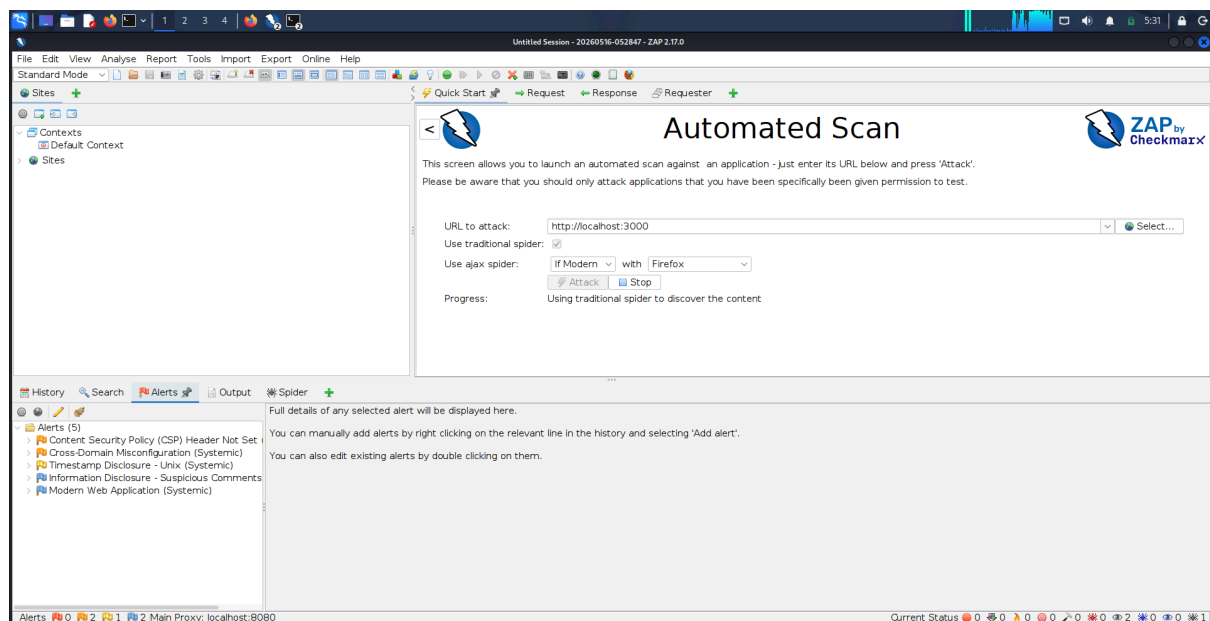
<http://localhost:3000>

Vulnerabilities Identified

- Content Security Policy (CSP) Header Not Set
- Cross-Domain Misconfiguration
- Missing Anti-clickjacking Header
- Session ID in URL Rewrite
- Private IP Disclosure
- Timestamp Disclosure – Unix
- X-Content-Type-Options Header Missing
- Information Disclosure – Suspicious Comments
- Modern Web Application

Screenshot Placeholder

The screenshot of zap alerts are shown below in fig 3



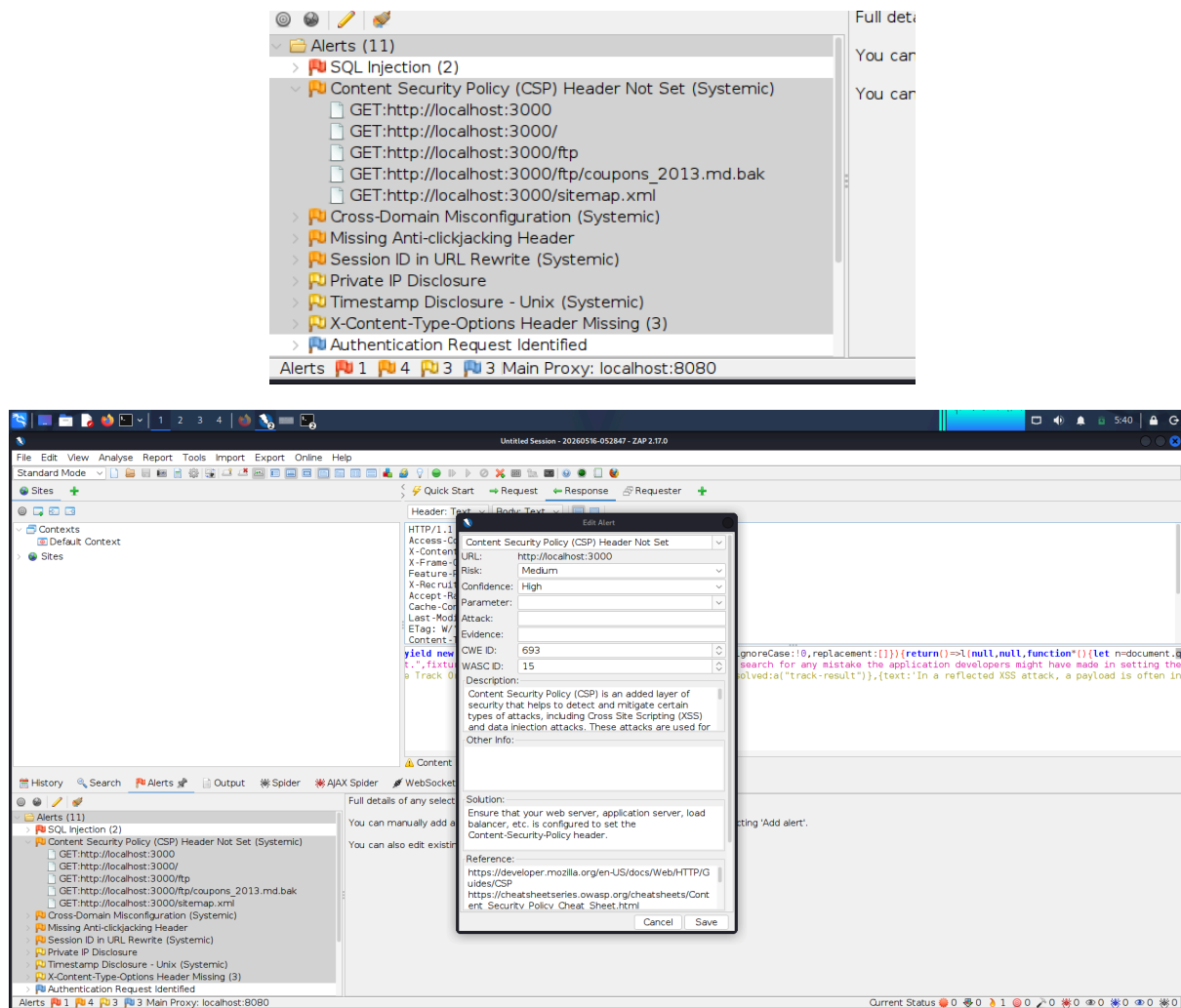


Fig 3 ZAP

7- Recommendations

The following recommendations are suggested to improve application security:

- Implement Content Security Policy (CSP).
- Sanitize and validate all user inputs.
- Use secure HTTP security headers.
- Prevent clickjacking using X-Frame-Options.
- Use parameterized SQL queries.
- Improve application error handling.
- Remove sensitive comments and debugging information.
- Store session identifiers securely in cookies.
- Regularly perform vulnerability assessments.

8. Conclusion

The security assessment successfully identified multiple common web application security weaknesses including missing security headers, information disclosure issues, and potential Cross-Site Scripting behavior.

Manual testing and OWASP ZAP scanning provided valuable insights into the application's security posture. Recommendations were provided to strengthen the application's overall security.

This assessment helped develop practical understanding of web application vulnerability assessment, OWASP Top 10 risks, and basic penetration testing methodologies.